

Лабораторная работа №4 Реализация SQL-запросов базы данных	Ф. И. О.	Шингарева Е. И.
	Группа	ПриИн-268
	Преподаватель	Аль-Мерри-Ганс Мохаммед Салех
	Дата сдачи	

Цель работы: Практическое освоение языка SQL путём написания не менее 100 разнообразных запросов к спроектированной базе данных, охватывающих все основные операции реляционной алгебры и возможности СУБД PostgreSQL.

Раздел	Запросов
1. Функциональные требования	10
2. UPDATE	7
3. DELETE	7
4. SELECT	16
5. LIKE и строки	7
6. INSERT SELECT	3
7. JOIN	16
8. GROUP BY	16
9. UNION / EXCEPT / INTERSECT	5
10. Вложенные SELECT	5
11. STRING_AGG и функции	3
12. WITH (CTE)	3
13. Строковые, датные, арифм. функции	7
14. Сложные запросы	5
ИТОГО	110

РАЗДЕЛ 1. Запросы по функциональным требованиям (10 запросов)

ФТ-1. [Оперативный] Поиск товаров Fender в наличии

Найти все товары бренда Fender, которые есть на складе. Ожидаем список с ценой, категорией и остатком.

SELECT

p.name AS наименование,
 b.name AS бренд,
 c.name AS категория,
 p.retail_price AS цена,

```

    p.quantity_in_stock AS остаток
FROM product p
    INNER JOIN brand b ON b.id = p.brand_id
    INNER JOIN category c ON c.id = p.category_id
WHERE b.name = 'Fender'
    AND p.quantity_in_stock > 0
ORDER BY p.retail_price;

```

ФТ-2. [Оперативный] Карточка клиента по номеру телефона

Вывести полную информацию о клиенте по уникальному номеру телефона.

```

SELECT
    c.last_name      AS фамилия,
    c.first_name     AS имя,
    c.middle_name    AS отчество,
    c.phone           AS телефон,
    c.email            AS email,
    c.registration_date AS дата_регистрации
FROM customer c
WHERE c.phone = '+79161234567';

```

ФТ-3. [Оперативный] Остатки синтезаторов Yamaha на складе

Узнать количество и минимальный запас всех синтезаторов бренда Yamaha.

```

SELECT
    p.name          AS товар,
    b.name           AS бренд,
    p.quantity_in_stock AS остаток_на_складе,
    p.minimum_stock  AS минимальный_запас
FROM product p
    INNER JOIN brand b ON b.id = p.brand_id
WHERE b.name = 'Yamaha'
    AND p.category_id = (
        SELECT id FROM category WHERE name = 'Клавишные'
    );

```

ФТ-4. [Оперативный] Поставщик товаров бренда Gibson

Получить контактные данные поставщиков, от которых поступали товары бренда Gibson.

```

SELECT DISTINCT
    s.company_name AS поставщик,
    s.contact_person AS контактное_лицо,
    s.phone         AS телефон,
    s.email          AS email,
    s.address        AS адрес
FROM supplier s
    INNER JOIN supply sup ON sup.supplier_id = s.id
    INNER JOIN supply_item si ON si.supply_id = sup.id
    INNER JOIN product p ON p.id = si.product_id
    INNER JOIN brand b ON b.id = p.brand_id
WHERE b.name = 'Gibson';

```

ФТ-5. [Оперативный] История продаж продавца Волкова Андрея

Посмотреть все продажи сотрудника, оформленные конкретным продавцом.

```

SELECT
    s.datetime AS дата_и_время,
    s.total_amount AS сумма,
    c.last_name AS клиент
FROM sale s
    LEFT JOIN customer c ON c.id = s.customer_id
    INNER JOIN employee e ON e.id = s.employee_id
WHERE e.last_name = 'Волков'
    AND e.first_name = 'Андрей'
ORDER BY s.datetime DESC;

```

ФТ-6. [Аналитический] Топ-10 продаваемых товаров за период

Вывести 10 самых популярных товаров по количеству проданных единиц. Фильтр задан датой начала тестовых данных.

```
SELECT
    p.name                AS товар,
    SUM(si.quantity)       AS продано_штук,
    SUM(si.quantity * si.price_at_sale) AS выручка
FROM sale_item si
    INNER JOIN product p ON p.id = si.product_id
    INNER JOIN sale     s ON s.id = si.sale_id
WHERE s.datetime >= '2024-06-01'
GROUP BY p.name
ORDER BY продано_штук DESC
LIMIT 10;
```

ФТ-7. [Аналитический] Выручка по категориям

Рассчитать суммарную выручку и количество проданных единиц по каждой категории за весь период.

```
SELECT
    c.name                AS категория,
    COUNT(DISTINCT si.sale_id) AS количество_продаж,
    SUM(si.quantity)       AS продано_единиц,
    SUM(si.quantity * si.price_at_sale) AS выручка
FROM sale_item si
    INNER JOIN product p ON p.id = si.product_id
    INNER JOIN category c ON c.id = p.category_id
    INNER JOIN sale     s ON s.id = si.sale_id
WHERE s.datetime >= '2024-06-01'
GROUP BY c.name
ORDER BY выручка DESC;
```

ФТ-8. [Аналитический] Динамика продаж по дням

Вывести число продаж и дневную выручку по дням за период тестовых данных.

```
SELECT
    DATE(s.datetime) AS день,
    COUNT(DISTINCT s.id) AS количество_продаж,
    SUM(s.total_amount) AS дневная_выручка
FROM sale s
WHERE s.datetime >= '2024-06-01'
GROUP BY DATE(s.datetime)
ORDER BY день;
```

ФТ-9. [Аналитический] Эффективность продавцов-консультантов

Сравнить число продаж, общую выручку и средний чек каждого продавца-консультанта.

```
SELECT
    e.last_name || ' ' || e.first_name AS сотрудник,
    e.position                        AS должность,
    COUNT(s.id)                      AS количество_продаж,
    SUM(s.total_amount)              AS общая_сумма_продаж,
    ROUND(AVG(s.total_amount), 2)    AS средний_чек
FROM sale s
    INNER JOIN employee e ON e.id = s.employee_id
WHERE e.position = 'Продавец-консультант'
GROUP BY e.id, e.last_name, e.first_name, e.position
ORDER BY общая_сумма_продаж DESC;
```

ФТ-10. [Аналитический] Товары ниже минимального запаса

Найти товары с дефицитом на складе. Добавлен DISTINCT ON, чтобы избежать дублирования (один товар от нескольких поставщиков).

```
SELECT DISTINCT ON (p.id)
```

```

p.name          AS товар,
p.quantity_in_stock AS остаток,
p.minimum_stock  AS минимум,
p.minimum_stock - p.quantity_in_stock AS дефицит,
b.name          AS бренд,
s.company_name   AS поставщик,
s.contact_person AS контакт
FROM product p
LEFT JOIN brand b ON b.id = p.brand_id
LEFT JOIN supply_item si ON si.product_id = p.id
LEFT JOIN supply sup ON sup.id = si.supply_id
LEFT JOIN supplier s ON s.id = sup.supplier_id
WHERE p.quantity_in_stock < p.minimum_stock
ORDER BY p.id, дефицит DESC;

```

РАЗДЕЛ 2. UPDATE — обновление данных (7 запросов)

UPD-1. Пересчёт складских остатков из поставок

Установить quantity_in_stock равным суммарному количеству поставленного товара. Выполняется после заполнения supply_item.

```

UPDATE product p
SET quantity_in_stock = COALESCE(
    (SELECT SUM(si.quantity)
     FROM supply_item si
     WHERE si.product_id = p.id),
    0
);

```

UPD-2. Пересчёт итоговой суммы продажи из позиций

Пересчитать total_amount в таблице sale как сумму произведений (количество × цена) всех позиций.

```

UPDATE sale s
SET total_amount = (
    SELECT SUM(si.quantity * si.price_at_sale)
    FROM sale_item si
    WHERE si.sale_id = s.id
);

```

UPD-3. Исправление email клиента Иванова Ивана

Исправить email клиента, введенный с опечаткой. Поиск по имени, а не по id.

```

UPDATE customer
SET email = 'ivan.ivanov@mail.ru'
WHERE last_name = 'Иванов'
AND first_name = 'Иван';

```

UPD-4. Повышение цены на 10% для товаров Fender

Увеличить розничную цену всех товаров бренда Fender на 10%.

```

UPDATE product
SET retail_price = ROUND(retail_price * 1.10, 2)
WHERE brand_id = (SELECT id FROM brand WHERE name = 'Fender');

```

UPD-5. Перевод принятых заказов в статус «Подтверждён»

Изменить статус заказов «Принят», оформленных более 2 дней назад.

```

UPDATE customer_order
SET status = 'Подтверждён'
WHERE status = 'Принят'
AND order_datetime < NOW() - INTERVAL '2 days';

```

UPD-6. Обновление контактного лица поставщика «Звук-Сервис»

Исправить контактное лицо и телефон конкретного поставщика. Поиск по названию компании.

```
UPDATE supplier
SET contact_person = 'Алина Кравцова',
    phone = '+74954500000'
WHERE company_name = 'ООО "Звук-Сервис";
```

UPD-7. Повышение минимального запаса для аксессуаров

Установить minimum_stock = 15 для товаров категории «Аксессуары», если текущий минимум меньше.

```
UPDATE product
SET minimum_stock = 15
WHERE category_id = (SELECT id FROM category WHERE name = 'Аксессуары')
AND minimum_stock < 15;
```

РАЗДЕЛ 3. DELETE — удаление данных (7 запросов)

PREP. Добавление тестовых данных для демонстрации DELETE

Вставляем временные записи, которые будут удалены следующими запросами.

```
INSERT INTO category (name, description)
VALUES ('ТЕСТ_УДАЛИТЬ', 'Временная тестовая категория');

INSERT INTO brand (name, country)
VALUES ('ТЕСТ_БРЕНД', 'Нигде');

INSERT INTO customer (last_name, first_name, phone, email, registration_date)
VALUES ('Тестов', 'Тест', '+70000000000', 'test@test.com', '2000-01-01');

INSERT INTO review (customer_id, product_id, rating, comment, datetime)
VALUES (
    (SELECT id FROM customer WHERE phone = '+70000000000'),
    1, 1, 'Тест — удалить', NOW()
);
```

DEL-1. Удаление тестовой категории

Удалить временную категорию по её уникальному названию.

```
DELETE FROM category
WHERE name = 'ТЕСТ_УДАЛИТЬ';
```

DEL-2. Удаление тестового бренда

Удалить временный бренд по его уникальному названию.

```
DELETE FROM brand
WHERE name = 'ТЕСТ_БРЕНД';
```

DEL-3. Удаление тестового отзыва

Удалить тестовый отзыв по содержимому комментария.

```
DELETE FROM review
WHERE comment = 'Тест — удалить';
```

DEL-4. Удаление тестового клиента

Удалить тестового клиента по уникальному номеру телефона.

```
DELETE FROM customer
WHERE phone = '+70000000000';
```

DEL-5. Удаление выданных заказов старше 30 дней

Очистить архивные заказы со статусом «Выдан», оформлённые более 30 дней назад.

```
DELETE FROM customer_order
WHERE status      = 'Выдан'
AND order_datetime < NOW() - INTERVAL '30 days';
```

DEL-6. Удаление пустых отзывов с низкой оценкой

Удалить отзывы без текста комментария и с оценкой ниже 3.

```
DELETE FROM review
WHERE comment IS NULL
AND rating < 3;
```

DEL-7. Удаление поставок без позиций

Удалить записи поставок, к которым нет ни одной позиции в supply_item.

```
DELETE FROM supply
WHERE id NOT IN (
    SELECT DISTINCT supply_id
    FROM supply_item
);
```

РАЗДЕЛ 4. SELECT — выборка данных (16 запросов)

SEL-1. DISTINCT — уникальные страны брендов

Получить список уникальных стран-производителей без повторений.

```
SELECT DISTINCT country AS страна_производства
FROM brand
ORDER BY страна_производства;
```

SEL-2. WHERE + AND — дорогие товары в наличии

Найти товары дороже 50 000 руб., которые есть на складе.

```
SELECT name, retail_price, quantity_in_stock
FROM product
WHERE retail_price > 50000
AND quantity_in_stock > 0
ORDER BY retail_price DESC;
```

SEL-3. WHERE + OR — продавцы и кассиры

Вывести сотрудников должности продавца-консультанта или кассира.

```
SELECT last_name, first_name, position, hire_date
FROM employee
WHERE position = 'Продавец-консультант'
OR position = 'Кассир';
```

SEL-4. WHERE + NOT — все, кроме администраторов

Вывести всех сотрудников, за исключением администраторов.

```
SELECT last_name, first_name, position
FROM employee
WHERE NOT position = 'Администратор'
ORDER BY last_name;
```

SEL-5. IN — товары двух категорий

Найти все товары категорий «Электрогитары» или «Клавишные».

```
SELECT
    p.name      AS товар,
    p.retail_price AS цена,
```

```
c.name      AS категория
FROM product p
  INNER JOIN category c ON c.id = p.category_id
WHERE c.name IN ('Электрогитары', 'Клавишные')
ORDER BY p.retail_price DESC;
```

SEL-6. BETWEEN — товары в ценовом диапазоне

Найти товары с ценой от 20 000 до 100 000 руб. включительно.

```
SELECT name, retail_price
FROM product
WHERE retail_price BETWEEN 20000 AND 100000
ORDER BY retail_price;
```

SEL-7. BETWEEN + дата — клиенты Q2 2024

Получить клиентов, зарегистрированных во втором квартале 2024 года (апрель–июнь).

```
SELECT last_name, first_name, registration_date
FROM customer
WHERE registration_date BETWEEN '2024-04-01' AND '2024-06-30'
ORDER BY registration_date;
```

SEL-8. IS NULL — анонимные продажи

Найти продажи, не привязанные ни к одному клиенту (анонимные покупки).

```
SELECT id AS id_продажи, datetime, total_amount
FROM sale
WHERE customer_id IS NULL;
```

SEL-9. AS — цена со скидкой 10% и экономия

Для каждого товара вычислить цену со скидкой 10% и сумму экономии покупателя.

```
SELECT
  name                AS наименование,
  retail_price        AS цена_руб,
  ROUND(retail_price * 0.9, 2) AS цена_со_скидкой_10_проц,
  retail_price - ROUND(retail_price * 0.9, 2) AS экономия_руб
FROM product
ORDER BY экономия_руб DESC;
```

SEL-10. Число дней с момента регистрации клиента

Вычислить, сколько дней прошло с момента регистрации каждого клиента.

```
SELECT
  last_name || ' ' || first_name AS клиент,
  registration_date             AS дата_регистрации,
  CURRENT_DATE - registration_date AS дней_с_регистрации
FROM customer
ORDER BY дней_с_регистрации DESC;
```

SEL-11. Коэффициент запаса (исправлено: CAST для точного деления)

Найти товары, остаток которых в 2+ раза больше минимума. Исправление: добавлен ::NUMERIC для точного деления.

```
SELECT
  name,
  quantity_in_stock AS остаток,
  minimum_stock     AS минимум,
  ROUND(
    quantity_in_stock::NUMERIC
    / NULLIF(minimum_stock, 0),
    2
  ) AS коэф_запаса
```

```
FROM product
WHERE quantity_in_stock >= minimum_stock * 2
ORDER BY коэф_запаса DESC;
```

SEL-12. DISTINCT + WHERE — должности сотрудников 2024 года

Вывести уникальные должности сотрудников, принятых в 2024 году.

```
SELECT DISTINCT position AS должность
FROM employee
WHERE hire_date >= '2024-01-01';
```

SEL-13. COALESCE — клиенты без email

Найти клиентов без email; вместо NULL показать строку «не указан».

```
SELECT
    last_name,
    first_name,
    phone,
    COALESCE(email, 'не указан') AS email
FROM customer
WHERE email IS NULL
    OR email = '';
```

SEL-14. Арифметика — наценка в процентах

Рассчитать торговую наценку в % от закупочной цены для каждого товара.

```
SELECT
    p.name AS товар,
    p.retail_price AS розница,
    si.purchase_price AS закупка,
    ROUND(
        (p.retail_price - si.purchase_price)
        / si.purchase_price * 100, 1
    ) AS наценка_проц
FROM product p
    INNER JOIN supply_item si ON si.product_id = p.id
ORDER BY наценка_проц DESC;
```

SEL-15. BETWEEN + дата — заказы июля 2024

Вывести заказы клиентов, оформленные в июле 2024 года.

```
SELECT
    co.id AS номер_заказа,
    c.last_name AS клиент,
    p.name AS товар,
    co.order_datetime AS дата,
    co.status AS статус
FROM customer_order co
    INNER JOIN customer c ON c.id = co.customer_id
    INNER JOIN product p ON p.id = co.product_id
WHERE co.order_datetime BETWEEN '2024-07-01' AND '2024-07-31 23:59:59'
ORDER BY co.order_datetime;
```

SEL-16. EXTRACT — поставки по месяцам

Извлечь год и месяц из даты поставки и подсчитать число поставок за каждый месяц.

```
SELECT
    EXTRACT(YEAR FROM supply_date) AS год,
    EXTRACT(MONTH FROM supply_date) AS месяц,
    COUNT(*) AS поставок_за_месяц
FROM supply
GROUP BY год, месяц
ORDER BY год, месяц;
```


РАЗДЕЛ 5. LIKE и работа со строками (7 запросов)

STR-1. LIKE — товары с «гитара» в названии

Найти товары, в названии которых содержится слово «гитара» (без учёта регистра).

```
SELECT name, retail_price
FROM product
WHERE LOWER(name) LIKE '%гитара%'
ORDER BY retail_price;
```

STR-2. LIKE — поставщики из городов на «М»

Найти поставщиков, адрес которых начинается с «г. М» (Москва и т.д.).

```
SELECT company_name, address
FROM supplier
WHERE address LIKE 'г. М%';
```

STR-3. LIKE — клиенты с gmail-адресом

Найти клиентов, у которых email заканчивается на @gmail.com.

```
SELECT last_name, first_name, email
FROM customer
WHERE email LIKE '%@gmail.com';
```

STR-4. NOT LIKE — бренды не из США

Вывести все бренды, страна которых не содержит «США».

```
SELECT name, country
FROM brand
WHERE country NOT LIKE '%США%';
```

STR-5. LIKE с _ — телефоны на +7916

Найти клиентов с телефоном, начинающимся на +7916.

```
SELECT last_name, first_name, phone
FROM customer
WHERE phone LIKE '+7916 _____';
```

STR-6. UPPER, LENGTH — ФИО в верхнем регистре

Вывести ФИО сотрудников заглавными буквами и длину фамилии.

```
SELECT
    UPPER(last_name || ' ' || first_name) AS ФИО_заглавными,
    LENGTH(last_name) AS длина_фамилии,
    position
FROM employee
ORDER BY длина_фамилии DESC;
```

STR-7. CONCAT + LIKE — описание синтезаторов

Найти товары, в описании которых есть слово «синтезатор», с полным наименованием.

```
SELECT
    CONCAT(b.name, ' ', p.name) AS полное_наименование,
    p.description
FROM product p
    INNER JOIN brand b ON b.id = p.brand_id
WHERE LOWER(p.description) LIKE '%синтезатор%';
```

РАЗДЕЛ 6. INSERT SELECT (3 запроса)

IS-0. Создание вспомогательных таблиц для INSERT SELECT

Создать две таблицы для хранения скопированных данных.

```
CREATE TABLE IF NOT EXISTS top_products_report (  
  product_name VARCHAR(255),  
  total_sold INT,  
  total_revenue DECIMAL(12,2),  
  avg_price DECIMAL(10,2)  
);  
  
CREATE TABLE IF NOT EXISTS low_stock_alert (  
  product_name VARCHAR(255),  
  current_stock INT,  
  minimum_stock INT,  
  deficit INT  
);
```

IS-1. INSERT SELECT — отчёт по продажам товаров

Скопировать агрегированные данные о продажах каждого товара в таблицу top_products_report.

```
INSERT INTO top_products_report  
  (product_name, total_sold, total_revenue, avg_price)  
SELECT  
  p.name,  
  SUM(si.quantity) AS total_sold,  
  SUM(si.quantity * si.price_at_sale) AS total_revenue,  
  AVG(si.price_at_sale) AS avg_price  
FROM sale_item si  
  INNER JOIN product p ON p.id = si.product_id  
GROUP BY p.name  
ORDER BY total_sold DESC;
```

IS-2. INSERT SELECT — товары с дефицитом

Скопировать список товаров с дефицитом на складе в таблицу low_stock_alert.

```
INSERT INTO low_stock_alert  
  (product_name, current_stock, minimum_stock, deficit)  
SELECT  
  name,  
  quantity_in_stock,  
  minimum_stock,  
  minimum_stock - quantity_in_stock AS deficit  
FROM product  
WHERE quantity_in_stock < minimum_stock;
```

IS-3. Проверка скопированных данных

Просмотреть содержимое обеих вспомогательных таблиц.

```
SELECT * FROM top_products_report ORDER BY total_sold DESC;  
  
SELECT * FROM low_stock_alert ORDER BY deficit DESC;
```

РАЗДЕЛ 7. JOIN — соединения таблиц (16 запросов)

JN-1. INNER JOIN — товары с категорией и брендом

Получить список всех товаров с названием категории и бренда.

```
SELECT  
  p.name AS товар,  
  c.name AS категория,  
  b.name AS бренд,
```

```

    p.retail_price AS цена
FROM product p
    INNER JOIN category c ON c.id = p.category_id
    INNER JOIN brand b ON b.id = p.brand_id
ORDER BY c.name, p.retail_price;

```

JN-2. LEFT JOIN — клиенты с числом и суммой покупок

Все клиенты, включая тех, кто ещё ничего не покупал (сумма = 0).

```

SELECT
    c.last_name || ' ' || c.first_name AS клиент,
    COUNT(s.id)                      AS количество_покупок,
    COALESCE(SUM(s.total_amount), 0) AS сумма_покупок
FROM customer c
    LEFT JOIN sale s ON s.customer_id = c.id
GROUP BY c.id, c.last_name, c.first_name
ORDER BY сумма_покупок DESC;

```

JN-3. LEFT JOIN — товары без продаж

Найти товары, которые ни разу не были проданы.

```

SELECT p.name, p.retail_price, p.quantity_in_stock
FROM product p
    LEFT JOIN sale_item si ON si.product_id = p.id
WHERE si.id IS NULL;

```

JN-4. RIGHT JOIN — сотрудники с числом продаж

Все сотрудники, включая тех, у кого нет ни одной продажи.

```

SELECT
    e.last_name || ' ' || e.first_name AS сотрудник,
    e.position,
    COUNT(s.id)                      AS продаж
FROM sale s
    RIGHT JOIN employee e ON e.id = s.employee_id
GROUP BY e.id, e.last_name, e.first_name, e.position
ORDER BY продаж DESC;

```

JN-5. FULL OUTER JOIN — клиенты и заказы

Полное внешнее соединение: все клиенты и все заказы, даже без связи.

```

SELECT
    c.last_name AS клиент,
    co.id       AS заказ,
    co.status   AS статус
FROM customer c
    FULL OUTER JOIN customer_order co ON co.customer_id = c.id
ORDER BY c.last_name NULLS LAST;

```

JN-6. M:N через sale_item — товары, купленные вместе

Найти пары товаров, которые покупали в одном чеке (связь M:N через sale_item).

```

SELECT
    p1.name AS товар_1,
    p2.name AS товар_2,
    COUNT(*) AS совместных_покупок
FROM sale_item si1
    INNER JOIN sale_item si2
        ON si2.sale_id = si1.sale_id
        AND si2.product_id > si1.product_id
    INNER JOIN product p1 ON p1.id = si1.product_id
    INNER JOIN product p2 ON p2.id = si2.product_id
GROUP BY p1.name, p2.name
ORDER BY совместных_покупок DESC;

```

JN-7. M:N через supply_item — число поставщиков на товар

Подсчитать, сколько разных поставщиков поставляли каждый товар.

```
SELECT
    p.name          AS товар,
    COUNT(DISTINCT sup.supplier_id) AS количество_поставщиков
FROM product p
    INNER JOIN supply_item si ON si.product_id = p.id
    INNER JOIN supply      sup ON sup.id       = si.supply_id
GROUP BY p.name
ORDER BY количество_поставщиков DESC;
```

JN-8. 4 таблицы — состав продаж с полной информацией

Каждая позиция продажи с датой, клиентом, продавцом, товаром и ценой.

```
SELECT
    s.datetime      AS дата,
    c.last_name     AS клиент,
    e.last_name     AS продавец,
    p.name          AS товар,
    si.quantity     AS кол_во,
    si.price_at_sale AS цена_продажи
FROM sale_item si
    INNER JOIN sale      s ON s.id = si.sale_id
    INNER JOIN product  p ON p.id = si.product_id
    LEFT JOIN customer  c ON c.id = s.customer_id
    INNER JOIN employee e ON e.id = s.employee_id
ORDER BY s.datetime;
```

JN-9. JOIN + агрегация — закупочная стоимость по поставщикам

Рассчитать суммарную стоимость всех поставок от каждого поставщика.

```
SELECT
    s.company_name AS поставщик,
    COUNT(DISTINCT sup.id) AS поставок,
    SUM(si.quantity * si.purchase_price) AS общая_закупочная_сумма
FROM supplier s
    INNER JOIN supply      sup ON sup.supplier_id = s.id
    INNER JOIN supply_item si ON si.supply_id    = sup.id
GROUP BY s.company_name
ORDER BY общая_закупочная_сумма DESC;
```

JN-10. JOIN — отзывы с именем клиента и товаром

Вывести все отзывы с именем покупателя и названием товара.

```
SELECT
    c.last_name || ' ' || c.first_name AS клиент,
    p.name          AS товар,
    r.rating        AS оценка,
    r.comment       AS комментарий,
    r.datetime      AS дата
FROM review r
    INNER JOIN customer c ON c.id = r.customer_id
    INNER JOIN product  p ON p.id = r.product_id
ORDER BY r.rating DESC;
```

JN-11. CROSS JOIN — все пары «категория × бренд»

Декартово произведение: все возможные пары категории и бренда (первые 20).

```
SELECT
    c.name AS категория,
    b.name AS бренд
FROM category c
```

```
CROSS JOIN brand b
ORDER BY c.name, b.name
LIMIT 20;
```

JN-12. INNER JOIN вместо NATURAL JOIN — состав поставок (исправлено)

Демонстрация INNER JOIN. Исправлено: NATURAL JOIN был удалён, так как обе таблицы имеют колонку id, что приводило к неверному соединению.

```
-- NATURAL JOIN некорректен здесь: таблицы имеют общую колонку id,
-- и JOIN происходил бы по id=id, а не по supply_id=id.
-- Исправлено на явный INNER JOIN:
SELECT
  si.id          AS позиция_id,
  si.quantity    AS количество,
  si.purchase_price AS закупочная_цена,
  sup.supply_date AS дата_поставки
FROM supply_item si
  INNER JOIN supply sup ON sup.id = si.supply_id
LIMIT 10;
```

JN-13. 3 таблицы — заказы с клиентом и товаром

Список заказов с именем клиента, его телефоном и информацией о товаре.

```
SELECT
  co.order_datetime AS дата_заказа,
  c.last_name || ' ' || c.first_name AS клиент,
  c.phone AS телефон,
  p.name AS товар,
  p.retail_price AS цена,
  co.status AS статус
FROM customer_order co
  INNER JOIN customer c ON c.id = co.customer_id
  INNER JOIN product p ON p.id = co.product_id
ORDER BY co.order_datetime DESC;
```

JN-14. LEFT JOIN — товары без отзывов

Найти товары, на которые ни один клиент не оставил отзыва.

```
SELECT p.name, p.retail_price
FROM product p
  LEFT JOIN review r ON r.product_id = p.id
WHERE r.id IS NULL;
```

JN-15. JOIN + HAVING — сотрудники с выручкой выше средней

Вывести сотрудников, суммарная выручка которых превышает среднее по всем продажам.

```
SELECT
  e.last_name || ' ' || e.first_name AS сотрудник,
  SUM(s.total_amount) AS итого_продаж
FROM sale s
  INNER JOIN employee e ON e.id = s.employee_id
GROUP BY e.id, e.last_name, e.first_name
HAVING SUM(s.total_amount) > (
  SELECT AVG(total_amount) FROM sale
)
ORDER BY итого_продаж DESC;
```

JN-16. 5 таблиц — товар, бренд, поставщик и цены

Связать каждый товар с брендом, поставщиком и закупочной ценой через 4 JOIN.

```
SELECT
  p.name AS товар,
  b.name AS бренд,
  s.company_name AS поставщик,
```

```

    si.purchase_price AS закупочная_цена,
    p.retail_price AS розничная_цена
FROM product p
  INNER JOIN brand b ON b.id = p.brand_id
  INNER JOIN supply_item si ON si.product_id = p.id
  INNER JOIN supply sup ON sup.id = si.supply_id
  INNER JOIN supplier s ON s.id = sup.supplier_id
ORDER BY p.name;

```

РАЗДЕЛ 8. GROUP BY + агрегатные функции (16 запросов)

GB-1. COUNT — товары в каждой категории

Подсчитать число товаров в каждой категории, включая пустые категории.

```

SELECT
  c.name AS категория,
  COUNT(p.id) AS количество_товаров
FROM category c
  LEFT JOIN product p ON p.category_id = c.id
GROUP BY c.name
ORDER BY количество_товаров DESC;

```

GB-2. SUM — выручка по продавцам

Рассчитать суммарную выручку каждого сотрудника от продаж.

```

SELECT
  e.last_name || ' ' || e.first_name AS продавец,
  SUM(s.total_amount) AS выручка
FROM sale s
  INNER JOIN employee e ON e.id = s.employee_id
GROUP BY e.id, e.last_name, e.first_name
ORDER BY выручка DESC;

```

GB-3. AVG + ROUND — средняя оценка товаров

Рассчитать среднюю оценку и число отзывов для каждого товара.

```

SELECT
  p.name AS товар,
  COUNT(r.id) AS отзывов,
  ROUND(AVG(r.rating), 2) AS средняя_оценка
FROM product p
  INNER JOIN review r ON r.product_id = p.id
GROUP BY p.name
ORDER BY средняя_оценка DESC;

```

GB-4. MAX + MIN + AVG — ценовой диапазон по категориям

Найти максимальную, минимальную и среднюю цену в каждой категории.

```

SELECT
  c.name AS категория,
  MAX(p.retail_price) AS максимум,
  MIN(p.retail_price) AS минимум,
  ROUND(AVG(p.retail_price), 2) AS среднее
FROM category c
  INNER JOIN product p ON p.category_id = c.id
GROUP BY c.name
ORDER BY максимум DESC;

```

GB-5. HAVING — категории с более чем 2 товарами

Вывести только те категории, в которых насчитывается более 2 товаров.

```

SELECT

```

```

    c.name AS категория,
    COUNT(p.id) AS товаров
FROM category c
    INNER JOIN product p ON p.category_id = c.id
GROUP BY c.name
HAVING COUNT(p.id) > 2
ORDER BY товаров DESC;

```

GB-6. LIMIT — топ-3 бренда по числу товаров

Найти три бренда с наибольшим количеством товаров в ассортименте.

```

SELECT
    b.name AS бренд,
    COUNT(p.id) AS товаров
FROM brand b
    INNER JOIN product p ON p.brand_id = b.id
GROUP BY b.name
ORDER BY товаров DESC
LIMIT 3;

```

GB-7. HAVING — поставщики с суммой поставок > 200 000

Найти поставщиков, суммарная стоимость поставок которых превышает 200 000 руб.

```

SELECT
    s.company_name AS поставщик,
    SUM(si.quantity * si.purchase_price) AS сумма_поставок
FROM supplier s
    INNER JOIN supply sup ON sup.supplier_id = s.id
    INNER JOIN supply_item si ON si.supply_id = sup.id
GROUP BY s.company_name
HAVING SUM(si.quantity * si.purchase_price) > 200000
ORDER BY сумма_поставок DESC;

```

GB-8. COUNT + GROUP BY — заказы по статусам

Подсчитать количество заказов клиентов в разбивке по статусам.

```

SELECT
    status AS статус,
    COUNT(id) AS количество
FROM customer_order
GROUP BY status
ORDER BY количество DESC;

```

GB-9. TO_CHAR + GROUP BY — выручка по месяцам

Рассчитать суммарную выручку и число продаж по каждому календарному месяцу.

```

SELECT
    TO_CHAR(datetime, 'YYYY-MM') AS месяц,
    SUM(total_amount) AS выручка,
    COUNT(id) AS продаж
FROM sale
GROUP BY TO_CHAR(datetime, 'YYYY-MM')
ORDER BY месяц;

```

GB-10. AVG + HAVING — бренды со средней ценой > 50 000

Найти бренды, средняя цена товаров которых превышает 50 000 руб.

```

SELECT
    b.name AS бренд,
    ROUND(AVG(p.retail_price), 2) AS средняя_цена
FROM brand b
    INNER JOIN product p ON p.brand_id = b.id
GROUP BY b.name
HAVING AVG(p.retail_price) > 50000

```

```
ORDER BY средняя_цена DESC;
```

GB-11. ORDER BY DESC + LIMIT — 5 самых дорогих товаров

Вывести пять самых дорогих товаров.

```
SELECT name, retail_price
FROM product
ORDER BY retail_price DESC
LIMIT 5;
```

GB-12. COUNT DISTINCT — клиенты с покупками

Подсчитать уникальное число клиентов, совершивших хотя бы одну покупку.

```
SELECT COUNT(DISTINCT customer_id) AS клиентов_с_покупками
FROM sale
WHERE customer_id IS NOT NULL;
```

GB-13. MAX — максимальная оценка по категориям

Найти наивысшую оценку отзывов для товаров каждой категории.

```
SELECT
    c.name      AS категория,
    MAX(r.rating) AS максимальная_оценка
FROM review r
    INNER JOIN product p ON p.id = r.product_id
    INNER JOIN category c ON c.id = p.category_id
GROUP BY c.name
ORDER BY максимальная_оценка DESC;
```

GB-14. Несколько агрегатов — статистика поставок по месяцам

Рассчитать число поставок, единиц товара и суммарную стоимость по каждому месяцу.

```
SELECT
    EXTRACT(MONTH FROM sup.supply_date) AS месяц,
    COUNT(DISTINCT sup.id)              AS поставок,
    SUM(si.quantity)                    AS единиц_получено,
    SUM(si.quantity * si.purchase_price) AS на_сумму
FROM supply sup
    INNER JOIN supply_item si ON si.supply_id = sup.id
GROUP BY EXTRACT(MONTH FROM sup.supply_date)
ORDER BY месяц;
```

GB-15. ORDER BY + LIMIT — топ-5 клиентов по сумме

Найти пять клиентов с наибольшей суммой покупок.

```
SELECT
    c.last_name || ' ' || c.first_name AS клиент,
    COUNT(s.id)                       AS покупок,
    SUM(s.total_amount)                AS итого
FROM customer c
    INNER JOIN sale s ON s.customer_id = c.id
GROUP BY c.id, c.last_name, c.first_name
ORDER BY итого DESC
LIMIT 5;
```

GB-16. Среднее число позиций в продаже

Рассчитать среднее количество позиций (строк) в одном чеке.

```
SELECT
    ROUND(AVG(cnt), 2) AS среднее_позиций_в_продаже
FROM (
    SELECT sale_id, COUNT(*) AS cnt
    FROM sale_item
```



```
GROUP BY sale_id  
) sub;
```

РАЗДЕЛ 9. UNION / EXCEPT / INTERSECT (5 запросов)

SET-1. UNION — все люди в системе (клиенты + сотрудники)

Объединить списки клиентов и сотрудников в один с указанием роли.

```
SELECT last_name, first_name, 'Клиент' AS роль FROM customer  
UNION  
SELECT last_name, first_name, 'Сотрудник' AS роль FROM employee  
ORDER BY last_name;
```

SET-2. UNION ALL — все телефоны клиентов и поставщиков

Объединить телефоны из двух таблиц, сохраняя дубли.

```
SELECT phone, 'Клиент' AS тип FROM customer WHERE phone IS NOT NULL  
UNION ALL  
SELECT phone, 'Поставщик' AS тип FROM supplier WHERE phone IS NOT NULL  
ORDER BY тип, phone;
```

SET-3. EXCEPT — клиенты без отзывов

Найти клиентов, которые ни разу не оставляли отзыв.

```
SELECT id, last_name, first_name FROM customer  
EXCEPT  
SELECT c.id, c.last_name, c.first_name  
FROM customer c  
INNER JOIN review r ON r.customer_id = c.id  
ORDER BY last_name;
```

SET-4. INTERSECT — клиенты, и покупавшие, и оставлявшие отзывы

Найти клиентов, которые и совершали покупки, и оставляли отзывы.

```
SELECT customer_id AS id FROM sale WHERE customer_id IS NOT NULL  
INTERSECT  
SELECT customer_id AS id FROM review  
ORDER BY id;
```

SET-5. UNION — все события клиента Иванова

Объединить историю покупок и отзывов конкретного клиента в единую хронологию.

```
SELECT  
  'Продажа' AS тип,  
  s.datetime AS дата,  
  s.total_amount::TEXT AS сумма_или_оценка  
FROM sale s  
INNER JOIN customer c ON c.id = s.customer_id  
WHERE c.last_name = 'Иванов' AND c.first_name = 'Иван'  
UNION  
SELECT  
  'Отзыв' AS тип,  
  r.datetime AS дата,  
  r.rating::TEXT AS сумма_или_оценка  
FROM review r  
INNER JOIN customer c ON c.id = r.customer_id  
WHERE c.last_name = 'Иванов' AND c.first_name = 'Иван'  
ORDER BY дата;
```

РАЗДЕЛ 10. Вложенные SELECT (5 запросов)

SUB-1. EXISTS — клиенты с активным заказом

Найти клиентов, у которых есть хотя бы один активный заказ.

```
SELECT last_name, first_name, phone
FROM customer c
WHERE EXISTS (
    SELECT 1
    FROM customer_order co
    WHERE co.customer_id = c.id
    AND co.status IN ('Принят', 'В пути', 'Подтверждён')
);
```

SUB-2. NOT EXISTS — товары без отзывов

Найти товары, на которые нет ни одного отзыва.

```
SELECT name, retail_price
FROM product p
WHERE NOT EXISTS (
    SELECT 1
    FROM review r
    WHERE r.product_id = p.id
);
```

SUB-3. ALL — товары дороже всех аксессуаров

Найти товары, цена которых выше цены любого товара из категории «Аксессуары».

```
SELECT name, retail_price
FROM product
WHERE retail_price > ALL (
    SELECT retail_price
    FROM product
    WHERE category_id = (SELECT id FROM category WHERE name = 'Аксессуары')
)
ORDER BY retail_price;
```

SUB-4. ANY — товары дешевле хоть одной электрогитары

Найти товары других категорий, цена которых ниже цены хотя бы одной электрогитары. Исправлено: IS DISTINCT FROM вместо <>.

```
SELECT name, retail_price
FROM product
WHERE retail_price < ANY (
    SELECT retail_price
    FROM product
    WHERE category_id = (SELECT id FROM category WHERE name = 'Электрогитары')
)
AND category_id IS DISTINCT FROM (
    SELECT id FROM category WHERE name = 'Электрогитары'
)
ORDER BY retail_price;
```

SUB-5. Вложенный SELECT + HAVING — бренды выше средней цены

Бренды, средняя цена товаров которых выше общей средней цены по всему ассортименту.

```
SELECT
    b.name AS бренд,
    ROUND(AVG(p.retail_price), 2) AS средняя_цена
FROM brand b
    INNER JOIN product p ON p.brand_id = b.id
GROUP BY b.name
HAVING AVG(p.retail_price) > (
    SELECT AVG(retail_price) FROM product
)
ORDER BY средняя_цена DESC;
```

РАЗДЕЛ 11. STRING_AGG и другие функции SQL (3 запроса)

AGG-1. STRING_AGG — список товаров каждой категории

Для каждой категории сформировать строку со всеми её товарами через запятую.

```
SELECT
    c.name                AS категория,
    STRING_AGG(p.name, ',' ORDER BY p.name) AS товары
FROM category c
    INNER JOIN product p ON p.category_id = c.id
GROUP BY c.name
ORDER BY c.name;
```

AGG-2. STRING_AGG — покупатели каждого товара (исправлено)

Для каждого товара собрать строку со всеми клиентами, которые его купили. Исправлено: DISTINCT убран из STRING_AGG, перенесён в подзапрос.

```
-- STRING_AGG(DISTINCT ..., ... ORDER BY ...) не поддерживается в PostgreSQL.
-- Исправлено: дедупликация выполняется в подзапросе.
SELECT
    товар,
    STRING_AGG(клиент, ',' ORDER BY клиент) AS покупатели
FROM (
    SELECT DISTINCT
        p.name                AS товар,
        c.last_name || ' ' || c.first_name AS клиент
    FROM product p
        INNER JOIN sale_item si ON si.product_id = p.id
        INNER JOIN sale      s  ON s.id = si.sale_id
        INNER JOIN customer  c  ON c.id = s.customer_id
) sub
GROUP BY товар
ORDER BY товар;
```

AGG-3. ARRAY_AGG — бренды по странам в виде массива

Сгруппировать бренды по стране и собрать их в массив PostgreSQL.

```
SELECT
    country                AS страна,
    ARRAY_AGG(name ORDER BY name) AS бренды,
    COUNT(*)               AS количество
FROM brand
GROUP BY country
ORDER BY количество DESC;
```

РАЗДЕЛ 12. Запросы с WITH (CTE) (3 запроса)

CTE-1. CTE + RANK — топ продаж с местом

С помощью CTE посчитать продажи по товарам, затем добавить ранг с помощью оконной функции RANK().

```
WITH product_sales AS (
    SELECT
        p.name                AS товар,
        SUM(si.quantity)      AS продано,
        SUM(si.quantity * si.price_at_sale) AS выручка
    FROM sale_item si
        INNER JOIN product p ON p.id = si.product_id
    GROUP BY p.name
)
SELECT
    товар,
    продано,
    выручка,
```

```

RANK() OVER (ORDER BY выручка DESC) AS место
FROM product_sales
ORDER BY место;

```

CTE-2. CTE — клиенты с категорией лояльности

Собрать статистику по каждому клиенту и присвоить категорию лояльности через CASE.

```

WITH customer_stats AS (
    SELECT
        c.id,
        c.last_name || ' ' || c.first_name AS клиент,
        COUNT(s.id) AS покупок,
        COALESCE(SUM(s.total_amount), 0) AS сумма
    FROM customer c
    LEFT JOIN sale s ON s.customer_id = c.id
    GROUP BY c.id, c.last_name, c.first_name
)
SELECT
    клиент,
    покупок,
    сумма,
    CASE
        WHEN покупок = 0 THEN 'Новый'
        WHEN покупок BETWEEN 1 AND 2 THEN 'Активный'
        ELSE 'Постоянный'
    END AS статус_клиента
FROM customer_stats
ORDER BY сумма DESC;

```

CTE-3. Два CTE — поставки и продажи в одном запросе

Использовать два CTE: данные о поставках и данные о продажах, объединить итоговым SELECT.

```

WITH supply_chain AS (
    SELECT
        s.company_name AS поставщик,
        sup.supply_date AS дата,
        si.quantity AS количество,
        si.purchase_price AS цена_закупки,
        p.name AS товар
    FROM supplier s
    INNER JOIN supply sup ON sup.supplier_id = s.id
    INNER JOIN supply_item si ON si.supply_id = sup.id
    INNER JOIN product p ON p.id = si.product_id
),
sales_data AS (
    SELECT
        p.name AS товар,
        SUM(si.quantity) AS продано
    FROM sale_item si
    INNER JOIN product p ON p.id = si.product_id
    GROUP BY p.name
)
SELECT
    sc.товар,
    sc.поставщик,
    sc.дата,
    sc.количество AS поставлено,
    COALESCE(sd.продано, 0) AS продано
FROM supply_chain sc
LEFT JOIN sales_data sd ON sd.товар = sc.товар
ORDER BY sc.товар, sc.дата;

```

FN-1. INITCAP, LEFT, REGEXP_REPLACE — форматирование клиентов

Вывести фамилию в формате Initcap, инициалы и телефон только из цифр.

```
SELECT
  INITCAP(last_name)          AS фамилия,
  LEFT(first_name, 1) || '.'
    || COALESCE(LEFT(middle_name, 1) || '.', '') AS инициалы,
  REGEXP_REPLACE(phone, '[^0-9]', '', 'g')      AS телефон_цифры
FROM customer;
```

FN-2. AGE + EXTRACT — стаж сотрудников

Рассчитать, сколько полных лет каждый сотрудник работает в компании.

```
SELECT
  last_name || ' ' || first_name AS сотрудник,
  hire_date                     AS принят,
  EXTRACT(YEAR FROM AGE(CURRENT_DATE, hire_date)) AS лет_в_компании
FROM employee
ORDER BY лет_в_компании DESC;
```

FN-3. TO_CHAR — читаемый формат даты продажи (исправлено)

Форматировать дату и время продажи в строку вида «01 Июня 2024 г., 14:30». Исправлено: 'г.' заключён в двойные кавычки.

```
SELECT
  id,
  TO_CHAR(datetime, 'DD Month YYYY "г.", HH24:MI') AS дата_продажи,
  total_amount
FROM sale
ORDER BY datetime;
```

FN-4. Арифметика — расчёт НДС 20%

Для каждого товара рассчитать цену без НДС и сумму самого НДС.

```
SELECT
  name          AS товар,
  retail_price  AS цена_с_НДС,
  ROUND(retail_price / 1.2, 2) AS цена_без_НДС,
  ROUND(retail_price - retail_price / 1.2, 2) AS НДС_20_проц
FROM product
ORDER BY retail_price DESC;
```

FN-5. CASE — ценовой сегмент товара

Присвоить каждому товару ценовой сегмент через условное выражение CASE.

```
SELECT
  name,
  retail_price,
  CASE
    WHEN retail_price < 10000 THEN 'Бюджетный'
    WHEN retail_price BETWEEN 10000 AND 60000 THEN 'Средний'
    WHEN retail_price > 60000 THEN 'Премиум'
  END AS ценовой_сегмент
FROM product
ORDER BY retail_price;
```

FN-6. DATE_TRUNC + INTERVAL — поставки за последние 3 месяца

Вывести поставки за последние 3 месяца с суммой и усечённым месяцем.

```
SELECT
  s.company_name AS поставщик,
  sup.supply_date AS дата,
  DATE_TRUNC('month', sup.supply_date) AS месяц,
  SUM(si.quantity * si.purchase_price) AS сумма
```

```
FROM supply sup
  INNER JOIN supplier s ON s.id = sup.supplier_id
  INNER JOIN supply_item si ON si.supply_id = sup.id
WHERE sup.supply_date >= CURRENT_DATE - INTERVAL '3 months'
GROUP BY s.company_name, sup.supply_date, DATE_TRUNC('month', sup.supply_date)
ORDER BY дата;
```

FN-7. NULLIF + ROUND — коэффициент оборачиваемости

Рассчитать отношение проданных единиц к текущему остатку. NULLIF защищает от деления на ноль.

```
SELECT
  name,
  quantity_in_stock,
  COALESCE(
    (SELECT SUM(si.quantity) FROM sale_item si WHERE si.product_id = product.id),
    0
  )
  AS продано,
  ROUND(
    COALESCE(
      (SELECT SUM(si.quantity) FROM sale_item si WHERE si.product_id = product.id),
      0
    )::NUMERIC
    / NULLIF(quantity_in_stock, 0),
    2
  )
  AS коэф_оборачиваемости
FROM product
ORDER BY коэф_оборачиваемости DESC NULLS LAST;
```

РАЗДЕЛ 14. Сложные запросы (5 запросов)

CMPLX-1. Полный отчёт по продажам: 5 JOIN + GROUP BY + HAVING + LIMIT

Сводный отчёт по каждому товару: категория, бренд, выручка, рейтинг, число продаж. Данные с 2024 года.

```
SELECT
  c.name AS категория,
  b.name AS бренд,
  p.name AS товар,
  SUM(si.quantity) AS продано_штук,
  SUM(si.quantity * si.price_at_sale) AS выручка,
  ROUND(AVG(r.rating), 2) AS рейтинг,
  COUNT(DISTINCT s.id) AS продаж
FROM sale_item si
  INNER JOIN product p ON p.id = si.product_id
  INNER JOIN category c ON c.id = p.category_id
  INNER JOIN brand b ON b.id = p.brand_id
  INNER JOIN sale s ON s.id = si.sale_id
  LEFT JOIN review r ON r.product_id = p.id
WHERE s.datetime >= '2024-01-01'
GROUP BY c.name, b.name, p.name
HAVING SUM(si.quantity) > 0
ORDER BY выручка DESC
LIMIT 10;
```

	AZ категория	AZ бренд	AZ товар	I23 продано_штук	I23 выручка	I23 рейтинг	I23 продаж
1	Студийное оборудование	Taylor	Профессиональный комплект #8694	144	15,408,311.79	4.67	6
2	Акустические гитары	Yamaha	Премиум набор #9945	150	14,221,458.8	2.4	4
3	Студийное оборудование	Shure	Премиум серия #7746	140	13,249,704.5	3.2	3
4	Электргитары	Epiphone	Премиум комплект #1927	136	12,504,769.68	2.75	4
5	Нотные издания	Epiphone	Классический инструмент #3011	128	11,601,806.96	2	5
6	Нотные издания	ESP	Компактный инструмент #2114	78	10,626,864.36	3.33	2
7	Духовые	Epiphone	Концертный модель #5399	132	10,410,200.8	2.25	4
8	Акустические гитары	Shure	Акустический модель #1690	152	10,116,490.04	3.5	6
9	Аксессуары	ESP	Концертный инструмент #7220	81	9,850,025.88	4.33	3
10	Ударные	Korg	Цифровой система #8493	104	9,692,305.88	2.25	5

CMPLX-2. Эффективность продавцов по категориям

Для каждого продавца и каждой категории: число продаж и выручка.

```
SELECT
    e.last_name || ' ' || e.first_name AS сотрудник,
    e.position AS должность,
    c.name AS категория,
    COUNT(DISTINCT s.id) AS продаж,
    SUM(si.quantity * si.price_at_sale) AS выручка_в_категории
FROM sale s
    INNER JOIN employee e ON e.id = s.employee_id
    INNER JOIN sale_item si ON si.sale_id = s.id
    INNER JOIN product p ON p.id = si.product_id
    INNER JOIN category c ON c.id = p.category_id
WHERE e.position LIKE '%продавец%'
    OR e.position LIKE '%Продавец%'
GROUP BY e.id, e.last_name, e.first_name, e.position, c.name
ORDER BY выручка_в_категории DESC;
```

	AZ сотрудник	AZ должность	AZ категория	123 продаж	123 выручка_в_категории
1	Соловьёв Денис	Продавец-консультант	Акустические гитары	309	149,996,763.45
2	Козлов Максим	Продавец-консультант	Аксессуары	298	143,839,812.97
3	Морозова Елена	Старший продавец	Акустические гитары	297	141,343,216.2
4	Волков Андрей	Продавец-консультант	Ударные	293	140,769,035.78
5	Волков Андрей	Продавец-консультант	Духовые	290	138,501,992.43

CMPLX-3. CTE — портрет покупателя

Сводный профиль каждого клиента: покупки, сумма, отзывы и средняя оценка.

```
WITH клиент_покупки AS (
    SELECT
        c.id,
        c.last_name || ' ' || c.first_name AS клиент,
        c.registration_date,
        COUNT(DISTINCT s.id) AS покупок,
        SUM(s.total_amount) AS потрачено,
        COUNT(DISTINCT r.id) AS отзывов,
        ROUND(AVG(r.rating), 2) AS ср_оценка
    FROM customer c
        LEFT JOIN sale s ON s.customer_id = c.id
        LEFT JOIN review r ON r.customer_id = c.id
    GROUP BY c.id, c.last_name, c.first_name, c.registration_date
)
SELECT *
FROM клиент_покупки
WHERE покупок > 0
ORDER BY потрачено DESC;
```

	123 id	AZ клиент	registration_date	123 покупок	123 потрачено	123 отзывов	123 ср_оценка
1	8,672	Лебедев Алексей	2024-06-01	6	7,849,347.75	5	2.4
2	518	Орлов Ольга	2022-08-24	5	7,040,266.55	5	3
3	9,253	Фёдоров Константин	2024-07-13	6	6,776,288.82	3	3.33
4	2,247	Волков Иван	2024-07-31	6	6,774,054.03	3	2.67
5	4,739	Новиков Константин	2023-11-24	4	6,587,592.15	5	3.8

CMPLX-4. Товары с активными заказами и малым остатком

Найти товары, на которые есть активные заказы, но которых мало на складе.

```
SELECT
    p.name AS товар,
    p.quantity_in_stock AS остаток,
    COUNT(co.id) AS активных_заказов,
    b.name AS бренд,
    s.company_name AS последний_поставщик
FROM customer_order co
    INNER JOIN product p ON p.id = co.product_id
    INNER JOIN brand b ON b.id = p.brand_id
    LEFT JOIN (
        SELECT DISTINCT ON (si.product_id)
            si.product_id,
            sup.supplier_id
        FROM supply_item si
    )
```

INNER JOIN supply sup ON sup.id = si.supply_id
ORDER BY si.product_id, sup.supply_date DESC
) last_sup ON last_sup.product_id = p.id
LEFT JOIN supplier s ON s.id = last_sup.supplier_id
WHERE co.status IN ('Принят', 'В пути', 'Подтверждён')
AND p.quantity_in_stock < 2
GROUP BY p.name, p.quantity_in_stock, b.name, s.company_name
ORDER BY активных_заказов DESC;

	A-Z товар	123 остаток	123 активных_заказов	A-Z бренд	A-Z последний_поставщик
1	Концертный набор #1068	1	1	Marshall	[NULL]
2	Премиум набор #1135	1	1	Taylor	[NULL]
3	Классический система #1143	0	1	Shure	[NULL]
4	Студийный серия #172	1	1	Yamaha	[NULL]
5	Профессиональный серия #1735	0	1	Ibanez	[NULL]
6	Цифровой серия #1742	1	1	Epiphone	[NULL]
7	Профессиональный комплект #1775	0	1	Sennheiser	[NULL]

CMPLX-5. Рейтинг поставщиков: объём, ассортимент, период

Полная аналитика по каждому поставщику: число поставок, ассортимент, объём и временной диапазон.

SELECT
s.company_name AS поставщик,
COUNT(DISTINCT sup.id) AS поставок,
COUNT(DISTINCT si.product_id) AS ассортимент_позиций,
SUM(si.quantity) AS единиц_поставлено,
SUM(si.quantity * si.purchase_price) AS на_сумму,
MIN(sup.supply_date) AS первая_поставка,
MAX(sup.supply_date) AS последняя_поставка
FROM supplier s
INNER JOIN supply sup ON sup.supplier_id = s.id
INNER JOIN supply_item si ON si.supply_id = sup.id
GROUP BY s.id, s.company_name
ORDER BY на_сумму DESC;

	A-Z поставщик	123 поставок	123 ассортимент_позиций	123 единиц_поставлено	123 на_сумму	первая_поставка	последняя_поставка
1	ООО "Музыкальный мир"	2	5	19	715,000	2024-05-10	2024-07-15
2	ИП "Гитара-Про"	2	3	35	269,000	2024-05-15	2024-07-20
3	ЗАО "Клавишные технологии"	1	2	5	254,000	2024-06-05	2024-06-05
4	ООО "Звук-Сервис"	1	2	4	211,000	2024-06-20	2024-06-20
5	Группа компаний "Драм-Бит"	1	2	11	160,000	2024-07-01	2024-07-01
6	ООО "Смычок"	1	1	50	7,500	2024-08-01	2024-08-01